

SET PLAYER AVATAR

View online: <https://www.construct.net/en/game-services/manuals/game-services/authentication/api-end-points/avatars/set-avatar>

Overview

Sets a players avatar, overwriting the existing one if the player already has an avatar set. Supported avatar file types are `Gif` , `Jpeg` , `Png` , `WebP` , `SVG` . The minimum avatar size is `32x32` pixels and the maximum avatar size is `2048x2048` pixels. The maximum file size of uploaded avatars depends on your CGS plan - please refer to your [CGS account](#) to find this value.

If a player is [restricted](#) from setting an avatar with the `Upload Avatar` restriction,, an error will be returned unless a game API key is used to authenticate the request.

Request URL

All parameters in the request must be sent as a `POST` HTTP request. Make all requests to the following URL:

```
https://auth.construct.net/setavatar.json
```

Authenticating The Request

This end point can be called by [signed in](#) players, or with [secret key](#) authentication.

Session Key Authentication

sessionKey string Required

The [session key](#) of the player you're making the request against.

Secret Key Authentication

secret string Required

Your games [secret API key](#).

playerID [guid](#) Required

The player ID you wish to make this request against.

Request Parameters

gameID **guid** Required

The ID of the game you are making this request against. You can find the ID of your game in your [Construct Game Services \(CGS\) account](#).

avatar string Sometimes required

The base64 encoded avatar picture being set for this player. You must specify this, `avatarURL` or `avatarData`.

avatarURL url (string) Sometimes required

The absolute URL of the avatar being set for this player. You must specify this, `avatar` or `avatarData`.

File Data

This request additionally supports the posting of files. These files should be sent as `multipart/form-data`, with the corresponding keys.

avatarData multipart/form-data Sometimes required

The data of the avatar image being set for this player. You must specify this, `avatar` or `avatarURL`.

Success Response

Successful responses always return a `HTTP 200` status code.

```
{
  "success": true,
  "avatars": [
    { ... },
    { ... }
  ],
  "formattingCulture": "en-us"
}
```

Response Properties

success **bool**

If the request was successful or not.

avatars Array<**Picture**>

A list of **picture objects** representing the players new avatar. Each picture object is the same avatar, but provided in different sizes. Sizes available are based on widths, and the widths `16, 32, 64, 96, 128, 256` will be shown here along with the original avatar size if it doesn't match a width in this list. Some sizes may not show if the width is less than the original avatar width.

formattingCulture string

If some return values are **culture specific**, this property indicates what culture the values have been rendered as.

Failure Response

Unsuccessful responses always return `HTTP 4xx` status codes.

```
{
  "success": false,
  "errorMessage": "The image provided is not valid Base64 data.",
  "helpURL": "https://www.construct.net/en/game-services/manuals/game-services/authentication/ap
  "shouldRetry": false
}
```

Response Properties

success **bool**

If the request was successful or not.

errorMessage string

An error message with more detailed information on why the request failed.

helpURL url (string)

A link to documentation which should provide help with the error.

shouldRetry **bool**

Should the client wait a short period of time and retry the request. Usually this is false, but returns true if the request failed due to rate limiting.